1. WHAT IS JAVASCRIPT?

JavaScript is a lightweight, dynamically-typed, multi-paradigm programming language of the web. It runs in every browser (via the V8, SpiderMonkey or JavaScriptCore engines) and on servers through Node.js. It powers interactivity, API calls, DOM manipulation, and full backend applications.

Adding JavaScript to a page

```
// 1. Inline (avoid — hard to maintain)
<button onclick="alert('Hi!')">Click</button>


// 2. Internal — inside a <script> tag
<script> console.log('Hello'); </script>


// 3. External file (best practice)
<script src="app.js"></script>
```

2. VARIABLES: let, const, var

let — block-scoped, value can be reassigned. **const** — block-scoped, binding cannot be reassigned (object contents can still change). **var** — legacy, function-scoped, hoisted as undefined. Prefer let/const; use const by default.

```
let score = 10;       // can be reassigned
const PI = 3.14;      // cannot be reassigned
var legacy = 'old';   // avoid in modern code


score = 20;           // OK
score = 'twenty';     // OK too — JS is dynamically typed
// PI = 3;            // TypeError: Assignment to constant variable
```

3. DATA TYPES

7 primitive types: string, number, bigint, boolean, undefined, null, symbol. Everything else is an object — arrays and functions are objects too.

```
typeof 'hello'        // 'string'
typeof 42             // 'number'  (integers & floats are one type)
typeof 42n            // 'bigint'
typeof true           // 'boolean'
typeof undefined      // 'undefined'
```

```
typeof null            // 'object'   <- historical bug in JS, never fixed!
typeof [1, 2, 3]       // 'object'   (arrays are objects)
typeof function(){}    // 'function'
```

4. OPERATORS

- Arithmetic: + - * / % (remainder) ** (power)
- Assignment: = += -= *= /= ??=
- Comparison: == (loose, coerces types) vs === (strict — always prefer)
- Logical: && (AND) || (OR) ! (NOT) ?? (nullish coalescing)
- Spread/rest: ... and optional chaining: ?.

```
5 + '5'     // '55'   (+ concatenates when either side is a string)
'5' == 5    // true   (== converts types before comparing)
'5' === 5   // false  (=== compares type too — use this)
0 == false  // true
0 === false // false


null ?? 'default'    // 'default' — ?? triggers only on null/undefined
0 || 'default'       // 'default' — || triggers on ALL falsy values
0 ?? 'default'       // 0         — 0 is not nullish, so kept
```

5. TYPE CONVERSION & FALSY VALUES

Explicit conversion uses Number(), String(), Boolean(), parseInt(). Falsy values (evaluate to false in conditions): false, 0, -0, 0n, '' (empty string), null, undefined, NaN. Everything else is truthy — including '0', [], and {}.

```
Number('42')       // 42
Number('4.2rem')   // NaN   (can't parse the whole string)
parseInt('42px')   // 42    (parses leading digits)
String(42)         // '42'
Boolean('')        // false
Boolean('0')       // true  (non-empty string is truthy)
Number.isNaN(NaN) // true  — NaN is not equal to anything, even itself
```

6. CONDITIONALS

```
const hour = 14;
```

```
if (hour < 12) {
  console.log('Good morning');
} else if (hour < 18) {
  console.log('Good afternoon');
} else {
  console.log('Good evening');
}

// switch — use break to stop fall-through
const fruit = 'mango';
switch (fruit) {
  case 'apple': console.log('red'); break;
  case 'mango': console.log('yellow'); break;
  default:      console.log('unknown');
}

// ternary operator — compact if/else
const ampm = hour < 12 ? 'AM' : 'PM';
```

7. LOOPS

```
for (let i = 0; i < 5; i++) console.log(i);

let n = 0;
while (n < 3) { n++; }          // condition checked first
do { n--; } while (n > 0);      // body runs at least once

// for...of — iterate VALUES of an array/string
for (const ch of 'abc') console.log(ch);   // a, b, c

// for...in — iterate KEYS of an object
for (const key in { a: 1, b: 2 }) console.log(key);  // a, b

[10, 20, 30].forEach((num, i) => console.log(i, num));
```

```
// break exits the loop; continue skips to next iteration
```

8. FUNCTIONS

Three ways to define: declaration (hoisted — callable before it appears), expression, and arrow function (concise, no own `this`). Parameters can have defaults; extra arguments can be collected with rest.

```
// Declaration — hoisted
function add(a, b) { return a + b; }


// Function expression
const multiply = function (a, b) { return a * b; };


// Arrow function (implicit return when no braces)
const square = n => n * n;


// Default + rest parameters
function greet(name = 'friend', ...others) {
  return `Hello ${name}, plus ${others.length} others`;
}
greet('Asha', 'B', 'C');   // "Hello Asha, plus 2 others"
greet();                   // "Hello friend, plus 0 others"
```

9. SCOPE, HOISTING & CLOSURES

Scope = where a variable is visible. Block scope ({}) applies to let/const; var ignores blocks. A closure is a function that remembers the variables from where it was created — even after that outer function has returned. This enables private state.

```
function counter() {
  let count = 0;                  // private variable
  return function () { return ++count; };
}
const next = counter();
next();   // 1
next();   // 2   <- closure still sees `count`
```

```
console.log(x);   // undefined  (var declarations are hoisted)
var x = 5;
// console.log(y); // ReferenceError — let is in the "temporal dead zone"
let y = 10;
```

10. ARRAYS

Basics & mutator methods

```
const nums = [1, 2, 3, 4, 5];
nums[0];              // 1  (zero-based index)
nums.push(6);         // add to end      → length 6
nums.pop();           // remove from end
nums.unshift(0);      // add to start
nums.shift();         // remove from start
nums.splice(1, 2);    // remove 2 items from index 1 — MUTATES
nums.slice(1, 3);     // copy from index 1 up to 3 — does NOT mutate
nums.indexOf(3);      // 2, or -1 if missing
nums.includes(3);     // true
```

Higher-order methods (the workhorses)

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2);              // [2, 4, 6, 8, 10] — transform each
nums.filter(n => n % 2 === 0);     // [2, 4]           — keep matches
nums.reduce((sum, n) => sum + n, 0); // 15             — fold to one value
nums.find(n => n > 2);             // 3  (first match, else undefined)
nums.some(n => n > 4);             // true (at least one matches)
nums.every(n => n > 0);            // true (all match)
[3, 1, 2].sort();                  // [1, 2, 3]  — MUTATES (use toSorted)
['a', 'b'].join('-');              // 'a-b'
[...nums, 6];                      // [1..6] — spread creates a new array
```

11. OBJECTS

```
const user = { name: 'Asha', age: 25, city: 'Pune' };
```

```
user.name;              // 'Asha'   (dot notation)
user['age'];            // 25       (bracket — needed for dynamic keys)
user.email = 'a@x.io'; // add
delete user.city;       // remove

const { name, age } = user;             // destructuring
const updated = { ...user, age: 26 };  // shallow copy + override

Object.keys(user);      // ['name', 'age', 'email']
Object.values(user);    // ['Asha', 25, 'a@x.io']
Object.entries(user);   // [['name','Asha'], ['age',25], ...]

user.profile?.bio;      // undefined — optional chaining, no crash

// methods & `this`
const account = {
  balance: 100,
  deposit(amount) { this.balance += amount; return this; },
};
account.deposit(50).balance;   // 150 — chaining via `return this`
```

12. STRINGS & TEMPLATE LITERALS

```
const name = 'Asha';
`Hello, ${name}! You have ${2 + 3} new messages.`  // template literal

'  hi  '.trim();            // 'hi'
'javascript'.length;        // 10
'javascript'.toUpperCase();// 'JAVASCRIPT'
'hello world'.includes('wor'); // true
'hello'.startsWith('he');       // true
'javascript'.slice(0, 4);       // 'java' (negative = from end)
'a,b,c'.split(',');             // ['a', 'b', 'c']
['a', 'b'].join('-');           // 'a-b'
```

```
'5'.padStart(2, '0');          // '05'

'a-b-c'.replaceAll('-', '_');  // 'a_b_c'
```

13. DOM MANIPULATION

The DOM is the browser's live tree of the HTML document. JavaScript selects, reads, and changes it.

```
// Selecting

const byId  = document.getElementById('save');

const one   = document.querySelector('.card');       // first match (CSS selector)

const many  = document.querySelectorAll('.item');    // NodeList of all


// Reading / writing content

one.textContent = 'Saved!';     // plain text — SAFE

// one.innerHTML = '<b>hi</b>'; // parses HTML — unsafe with user input (XSS)


// Styling & classes

byId.classList.add('active');

byId.classList.toggle('open');

byId.classList.remove('active');

byId.style.backgroundColor = 'crimson';


// Creating & inserting

const li = document.createElement('li');

li.textContent = 'New item';

document.querySelector('ul').append(li);

li.remove();
```

14. EVENTS

```
btn.addEventListener('click', function (event) {

  console.log(event.type);     // 'click'

  console.log(event.target);   // element that triggered it

});
```

```
form.addEventListener('submit', e => {
  e.preventDefault();          // stop the page reload
});

document.addEventListener('keydown', e => {
  if (e.key === 'Escape') closeModal();
});

// Event delegation — 1 listener handles many children
list.addEventListener('click', e => {
  if (e.target.matches('li')) e.target.remove();
});
// Common events: click, input, change, submit, keydown, mouseover, scroll
```

Events bubble upward from the target through its ancestors — that is what makes delegation possible. removeEventListener() needs the same function reference.

15. ASYNCHRONOUS JAVASCRIPT

JS is single-threaded: slow work (network, timers) is handed to the environment and results come back through callbacks, Promises, or async/await. Promises are either pending, fulfilled, or rejected.

```
setTimeout(() => console.log('later'), 1000);

// Promises — .then / .catch / .finally
fetch('https://api.example.com/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// async/await — same flow, written like sync code
async function loadData() {
  try {
    const res = await fetch('https://api.example.com/data');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
```

```
  } catch (err) {
    console.error('Failed:', err);
  }
}

// Parallel requests
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

16. ERROR HANDLING

```
try {
  riskyOperation();
} catch (err) {
  console.error(err.name);      // 'TypeError', 'ReferenceError', ...
  console.error(err.message);   // human-readable description
} finally {
  cleanup();                    // always runs, error or not
}

if (!input) throw new Error('Input is required');
// Common built-ins: TypeError, ReferenceError, SyntaxError, RangeError
```

17. CLASSES & OOP

```
class Animal {
  #secret = 'hidden';               // private field
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
  static isAnimal(obj) { return obj instanceof Animal; }
  get upperName() { return this.name.toUpperCase(); }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);                    // call parent constructor first
```

```
    this.breed = breed;
  }
  speak() { return `${this.name} barks`; }   // method override
}

const d = new Dog('Rocky', 'Labrador');
d.speak();                // 'Rocky barks'
d.upperName;              // 'ROCKY'
Animal.isAnimal(d);       // true
```

18. JSON & BROWSER STORAGE

localStorage persists across browser restarts (per origin); sessionStorage clears when the tab closes. Both store strings only — serialize objects with JSON.

```
const obj = { name: 'Asha', age: 25 };

const json = JSON.stringify(obj);  // '{"name":"Asha","age":25}'
const back = JSON.parse(json);     // { name: 'Asha', age: 25 }

localStorage.setItem('user', json);
const saved = localStorage.getItem('user');  // string or null
localStorage.removeItem('user');
localStorage.clear();              // wipe everything
sessionStorage.setItem('tmp', 'per-tab only');
```

19. QUICK INTERVIEW FACTS

- var vs let vs const — var is function-scoped and hoisted as undefined; let/const are block-scoped with a temporal dead zone; const blocks reassignment, not mutation.
- == vs === — == coerces types before comparing; === checks value AND type. Always use ===.
- null vs undefined — undefined means 'not assigned yet'; null is an intentional empty value.
- Arrow vs regular functions — arrows have no own this, are not hoisted, and cannot be constructors.
- Event loop — synchronous code runs first; microtasks (Promises) then macrotasks (timers) run when the call stack empties.
- Shallow vs deep copy — spread {...obj} copies one level; structuredClone(obj) deep-copies nested values.
- NaN !== NaN — test with Number.isNaN().

- Arrays are objects — typeof [] is 'object'; check with Array.isArray().

Happy coding! Keep this doc as a living reference — ask me to extend any section with exercises or deeper examples.